Universität Rostock



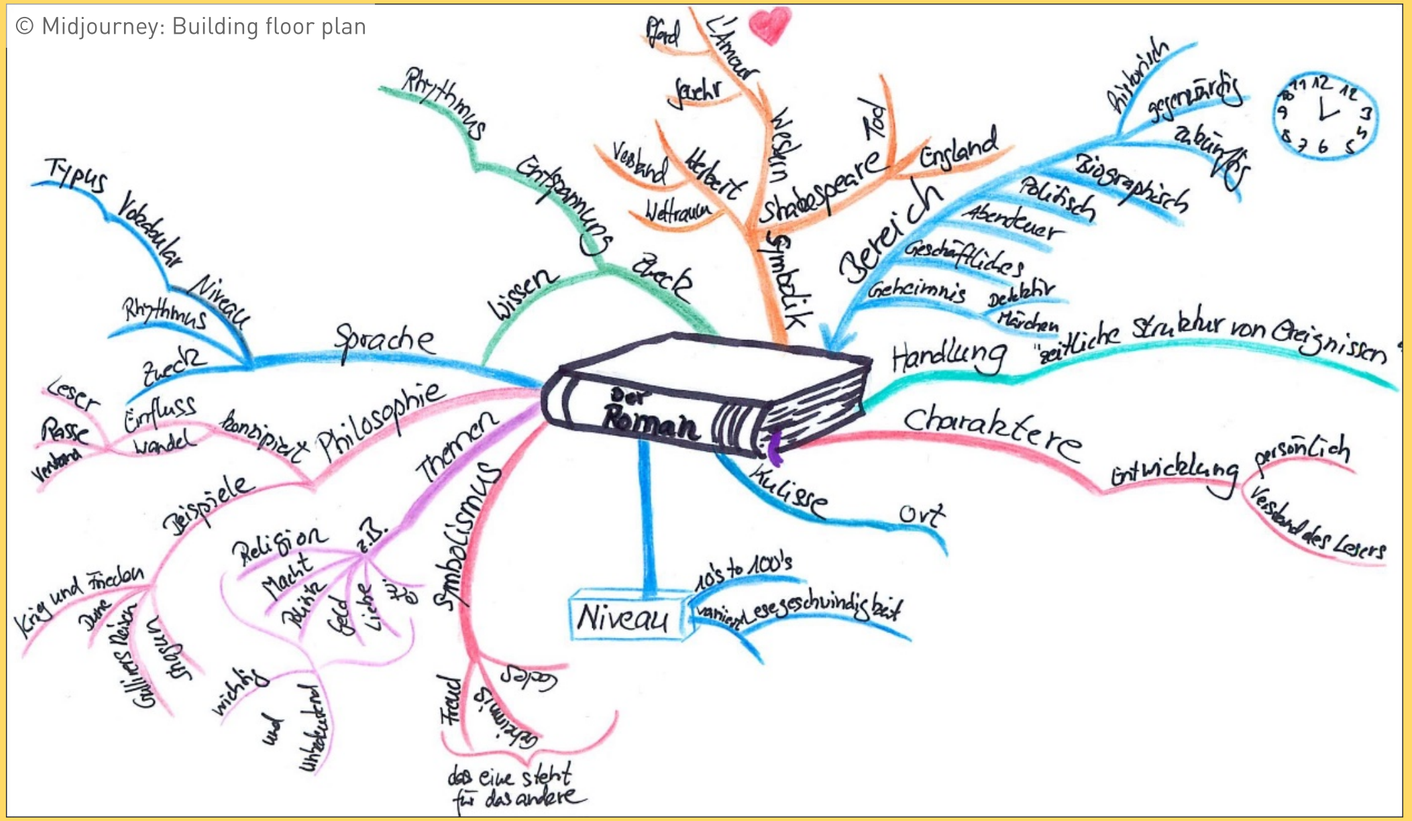
REKURSION

Joern Ploennigs · AI4SC

Midjourney: Fractal

Hörsaalfrage

Was müsst ihr tun, um alle Räume in einem mehrstöckigen Gebäude zu inventarisieren?



REKURSION

Rekursion ist eine Strategie für Probleme, die sich in kleinere Probleme **desselben Typs** zerlegen lassen. Eine rekursive Funktion ruft sich selbst auf und löst dabei jedes Mal ein einfacheres Teilproblem.

- Zwei Bestandteile: *Basisfall* (direktes Ergebnis) und *rekursiver Fall* (Zerlegung)
- Geeignet für **hierarchische Strukturen**: Bäume, Modelle, Gebiete
- → S09a: Rekursion als Divide-and-Conquer-Algorithmusstrategie

Das rekursive Muster

```python
def löse(problem):
    if problem.ist_klein_genug():       # Basisfall
        return direkte_lösung(problem)
    teile = zerlege(problem)            # Rekursiver Fall
    return kombiniere([löse(teil) for teil in teile])
```

- **Basisfall**: kein weiterer Selbstaufruf — direkte Rückgabe
- **Rekursiver Fall**: Teilung → Delegation → Zusammenführung

Beispiel: Fakultät (Einführung)

```python
def factorial(x):
    if x <= 1:                    # Basisfall
        return 1
    return x * factorial(x - 1)   # Rekursiver Fall
```

- `factorial(5)` → `5 × factorial(4)` → … → `5 × 4 × 3 × 2 × 1`
- Basisfall: `factorial(1)` → `1`

Achtung: Endlosrekursion

- Wenn der Basisfall nie erreicht wird, ruft die Funktion sich endlos selbst auf
- Deshalb **muss** mindestens ein Pfad ohne rekursiven Aufruf existieren
- Python stoppt bei ~1000 Aufrufen mit `RecursionError`

Zwei Speicherregionen im RAM

Stack

- Für Funktionsaufrufe
- LIFO-Prinzip (Last In, First Out)
- Begrenzter Speicher

Heap

- Für Variablen
- FIFO-ähnlich (First In, First Out)
- Dynamisch groß (RAM)

Berechnung Kleiner Gauß

$$s(n) = n + s(n - 1)$$

s(4) = 10

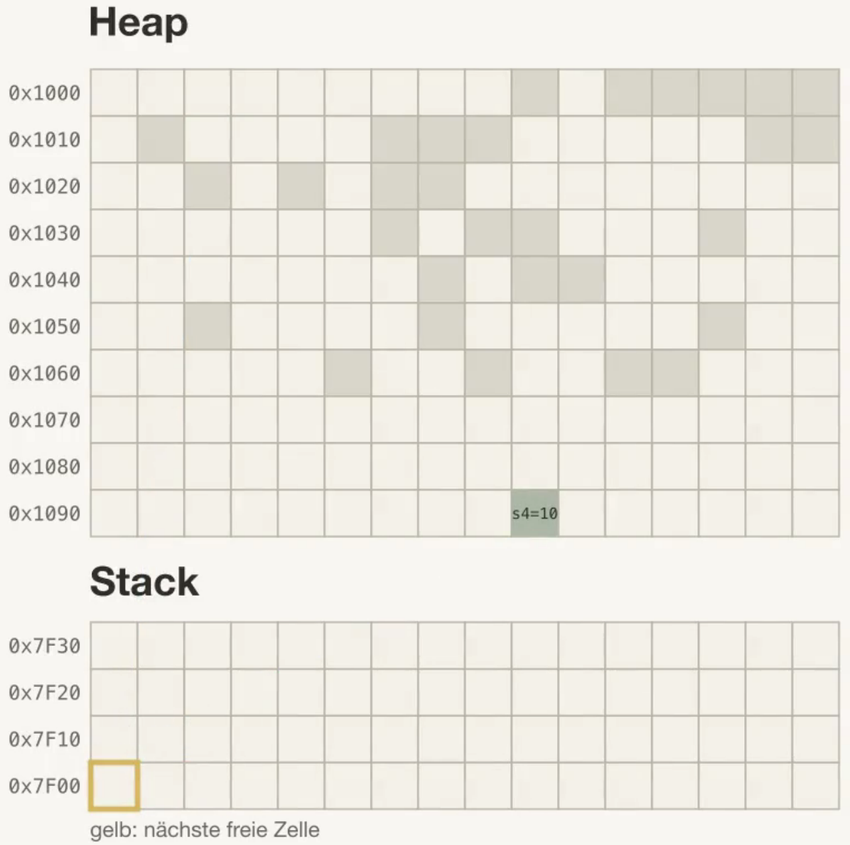
s(3) = 6

s(2) = 3

s(1) = 1

s(0) = 0

Ergebnis s(4) bleibt im Heap



Der Stack wächst mit jedem Aufruf

- Der Stack ist eine Liste in der nachvollzogen wird welche Funktionen gerade aufgerufen wurden
- Damit ist bekannt, wohin man zurückkehrt, wenn eine Funktion beendet wird
- Bei endloser Rekursion, wird die Liste unendlich groß
- Es kommt bei endloser Rekursion zu Stack-Overflow-Fehlern
- Es kommt zum harten Absturz des Programmes

Berechnung Kleiner Gauß

$$s(n) = n + s(n - 1)$$

ohne Abbruchbedingung $s(0) = 0$

Stack-Größe

0x7F30	n=−19	ret	n=−20	ret	n=−21	ret	n=−22	ret	n=−23	ret	n=−24	ret	n=−25	ret	n=−26	ret			
0x7F20	n=−11	ret	n=−12	ret	n=−13	ret	n=−14	ret	n=−15	ret	n=−16	ret	n=−17	ret	n=−18	ret			
0x7F10	n=−3	ret	n=−4	ret	n=−5	ret	n=−6	ret	n=−7	ret	n=−8	ret	n=−9	ret	n=−10	ret			
0x7F00	n=5	ret	n=4	ret	n=3	ret	n=2	ret	n=1	ret	n=0	ret	n=−1	ret	n=−2	ret			

`RecursionError: maximum recursion depth exceeded`

Der Stack ist voll: Stack Overflow

Agentic Workflow: Rekursive Funktion prüfen

Wenn KI eine rekursive Funktion generiert:

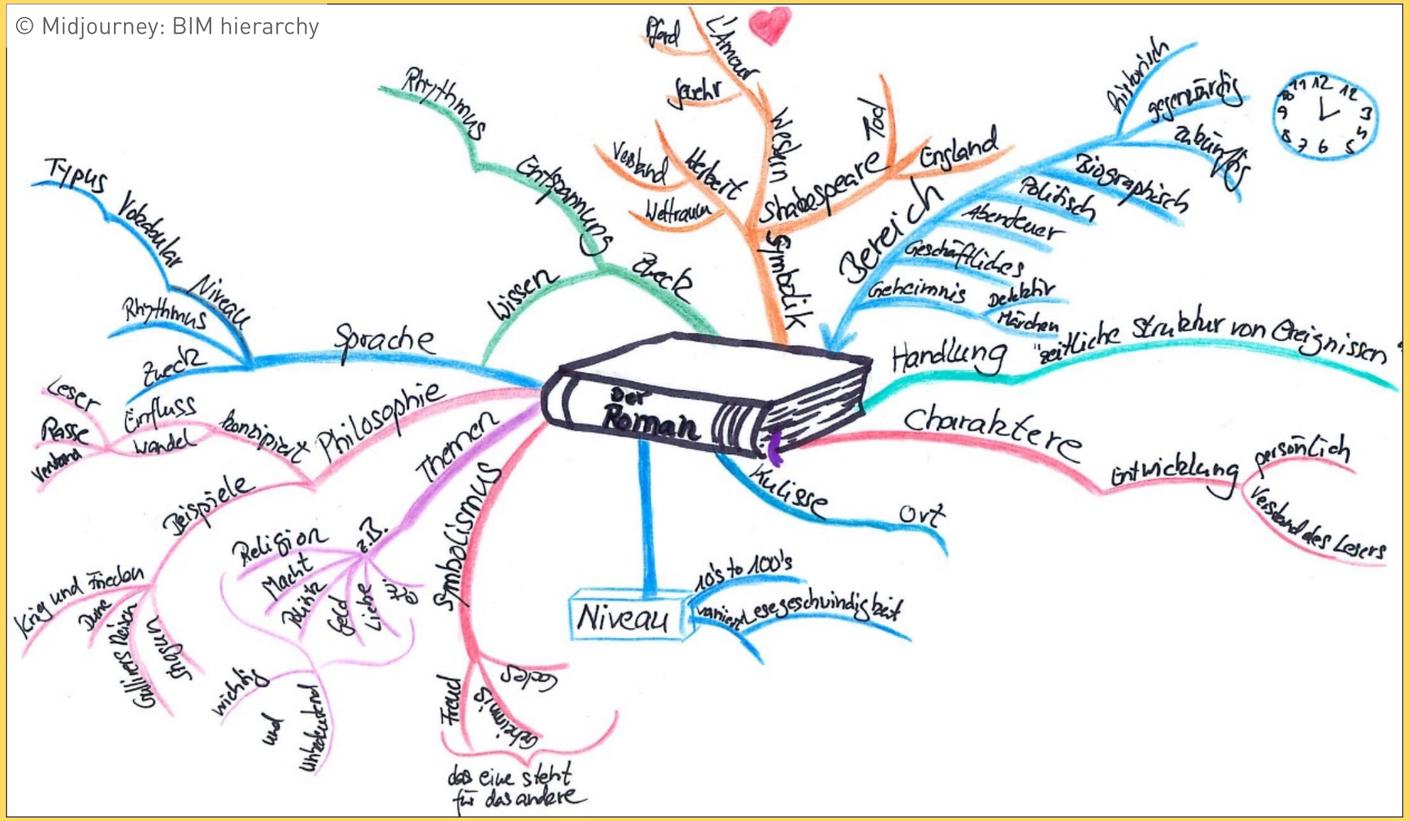
01 **Basisfall identifizieren** — gibt es mindestens einen Pfad ohne Selbstaufruf?

02 **Erste zwei Aufrufe von Hand durchlaufen** — stimmt das Ergebnis?

03 **Stack-Tiefe abschätzen** — wie tief rekursiert die Funktion bei realer Eingabegröße? (Python-Limit: ~1000)

Hörsaalfrage

Wie navigiert man durch alle Elemente eines BIM-Modells — ohne vorher die Tiefe zu kennen?



BIM-Hierarchie — natürliche Rekursion

Gebäudemodell

```
Gebäude A
├─ Stockwerk 1
│  ├─ Raum 101
│  │  ├─ Tür
│  │  └─ Fenster
│  └─ Raum 102
└─ Stockwerk 2
   └─ Raum 201
```

- Jede Ebene hat die **gleiche Struktur**
- Dieselbe Funktion traversiert jede Ebene
- Basisfall: Element ohne Kinder
- Tiefe ist zur Compile-Zeit unbekannt

KI-generierter Code: BIM-Traversierung

```python
# KI-generiert
def bim_traversiere(element, tiefe=0):
    print("  " * tiefe + element["name"])
    for kind in element.get("kinder", []):    # Rekursiver Fall
        bim_traversiere(kind, tiefe + 1)      # Basisfall: leere Liste
```

Vor dem Ausführen:

- Wo ist der Basisfall?
- Was passiert beim ersten rekursiven Aufruf?
- Wie tief kann die Hierarchie werden?

BIM: Erwartete Ausgabe verifizieren

Eingabe

```
Gebäude A
├─ Stockwerk 1
│  ├─ Raum 101 (Tür, Fenster)
│  └─ Raum 102
└─ Stockwerk 2
   └─ Raum 201
```

Erwartete Ausgabe

```
Gebäude A
  Stockwerk 1
    Raum 101
      Tür
      Fenster
    Raum 102
  Stockwerk 2
    Raum 201
```

Quadtree — Rekursive Raumaufteilung

01 Gebiet enthält mehr als einen Punkt

02 → Vierteile in vier Quadranten

03 Jeden Quadranten rekursiv unterteilen

Basisfall: ⩽ 1 Punkt im Quadranten oder Tiefenlimit

Anwendung in der Umweltinformatik:

- Geländemodelle
- Sensornetze
- GIS-Abfragen und Nachbarschaftssuche

Quadtree — Rekursiver Code

```python
def quadtree_unterteile(grenzen, punkte, max_tiefe=4, tiefe=0):
    if tiefe >= max_tiefe or len(punkte) <= 1:    # Basisfall
        return {"grenzen": grenzen, "punkte": punkte}
    mx, my = (grenzen[0]+grenzen[2])/2, (grenzen[1]+grenzen[3])/2
    quadranten = [...]   # vier Teilgebiete
    return [quadtree_unterteile(b, pts, ...) for b, pts in quadranten]
```

- Basisfall: Tiefenlimit oder ⩽ 1 Punkt
- Vier rekursive Aufrufe — einer je Quadrant
- Typische Tiefe: 4–8 für Geländedaten

Rekursion vs. Iteration — Vergleich

Rekursion

+ Klar bei hierarchischen Problemen
+ Entspricht natürlicher Problemstruktur
– Stack-Tiefe begrenzt (~1000 in Python)
– Schwerer zu debuggen

Iteration (Schleife)

+ Kein Stack-Overhead
+ Einfacher zu debuggen
– Expliziter Stack nötig für Baumtraversierung
– Unnatürlich für hierarchische Daten

Wann Rekursion wählen?

Ja — Rekursion

- Datenstruktur ist selbst rekursiv (BIM, Quadtree, Verzeichnisbaum)
- Rekursionstiefe bleibt klein und kontrollierbar
- Lesbarkeit hat Vorrang

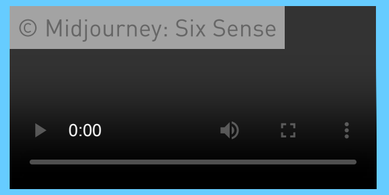
Nein — Iteration

- Lange flache Listen oder einfache Schleifen
- Tiefe unbekannt oder potenziell groß
- Performance-kritisch

> Die Grenze des Stacks gilt für jede Rekursion: Python stoppt bei rund 1000 Aufrufen mit `RecursionError`.

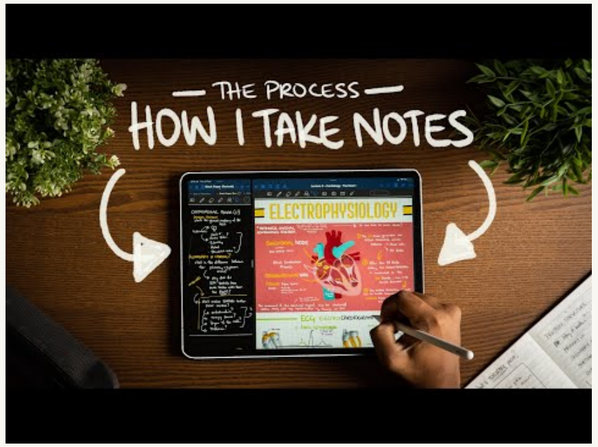
Lesson Learned

Welche Lerntypen gibt es?

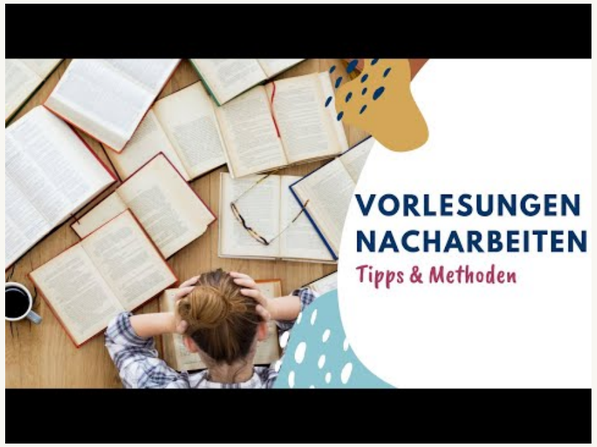


Lesson Learned - Nacharbeit

- **S**urvey (Überblick gewinnen): Blättert durch ein Thema und gewinnt einen Überblick
- **Q**uestion (Fragen): Notiert euch Fragen zu Punkten die ihr nicht versteht.
- **R**ead (Lesen): Lest eure Notizen durch; Hebt Schlüsselwörter hervor; Recherchiert zu den Fragen.
- **R**ecite (Wiedergeben): Rekapituliert jeden Abschnitt (Inhalt, Schlüsselwörter, Zusammenhänge). Strukturiert euch die Inhalte, z.B. mit Mindmaps. Wiederholt Gelerntes mehrmals (Karteikarten).
- **R**eview (Rückblick): Wiederholt gedanklich die wichtigsten Punkte und beantwortet eure Fragen.

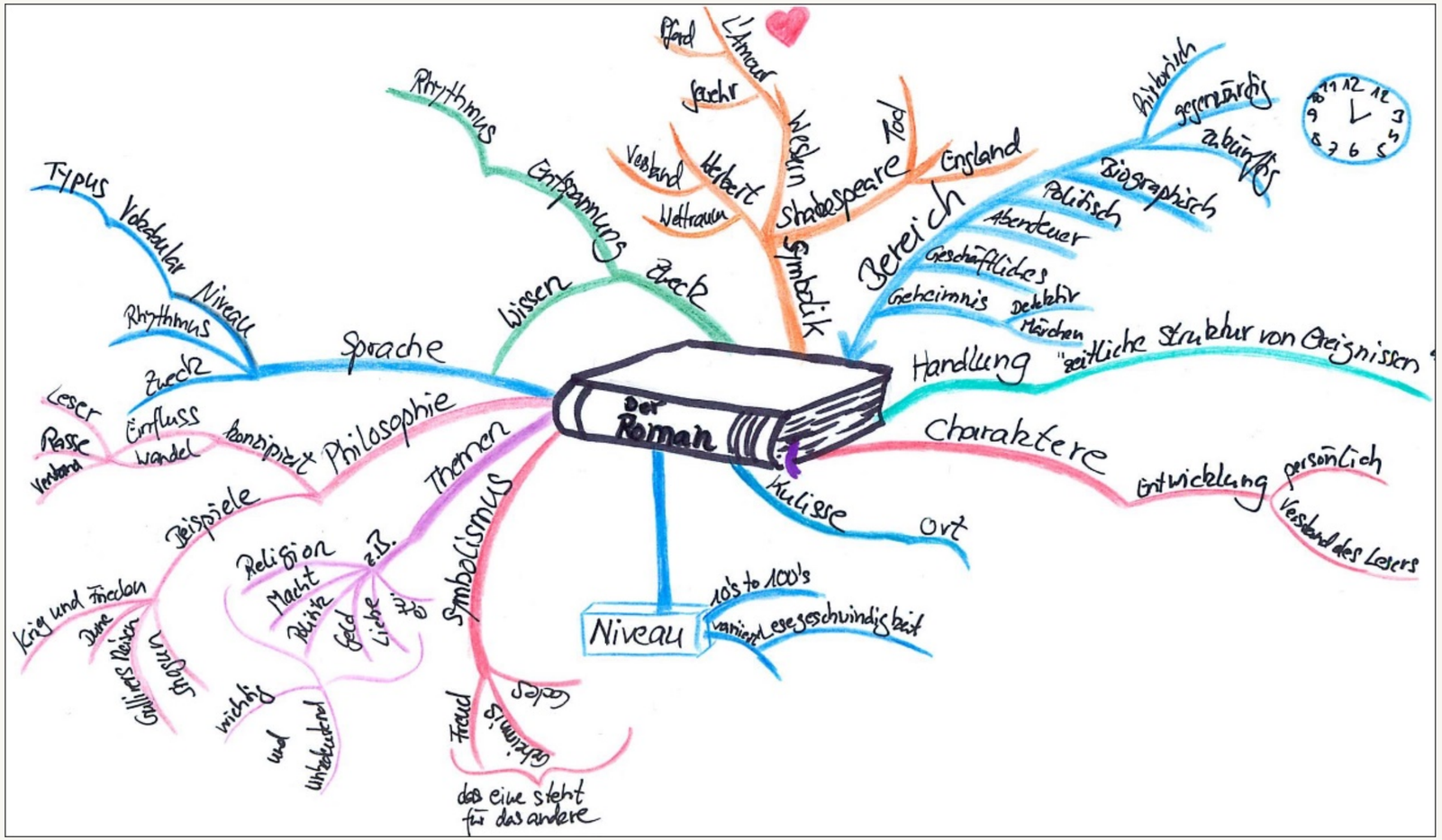


https://youtu.be/n0ql-yeY9u0



https://youtu.be/N0NST7mwhX8

Mindmap



- Man arbeitet von innen nach außen
- Reduziert Themen auf einzelne Begriffe
- Man entwickelt automatisch eine Struktur
- Nutzt Farben und Symbole
- Verweise zwischen Begriffen erschaffen

QUESTIONS